Help Menu

The commands are described under the **Help Menu**.

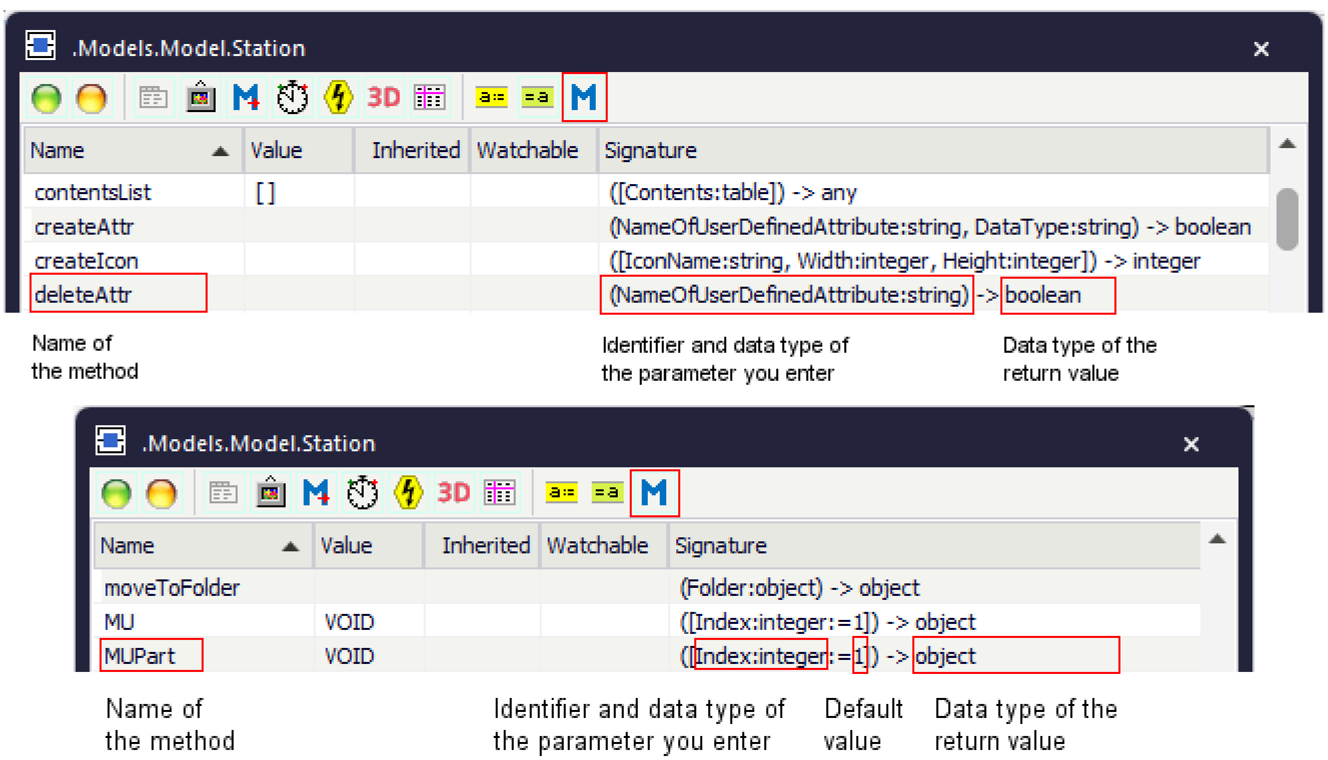
Methods of the Turntable

_Methods of the Turntable

The *Turntable* provides:

- The *methods* listed in the table of contents to the left.
- The **_Methods of Curved Objects**.
- The **Methods of the Material Flow Objects**.
- The **Methods of All Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.

- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

> **Note**
>
> Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, `:= false` in the example above.

If the *method* has a return value, the signature shows its data type after the arrow ->, `→ boolean` in the example above.

calculateAngles [SimTalk] - Turntable

Computes the angles at which you connected the *Turntable* designated by <Path> with its predecessor and with its successor in the *Frame*.

Remarks

The *Turntable* then enters the **Entry Angles** into the **Entry Angles Table** and the **Exit Angles** into the **Exit Angles Table**.

Type

Method

Syntax

```
<Path>.calculateAngles
```

Example

```
MyTurntable.calculateAngles
```

SimTalk

getEntryAngles [SimTalk]

getExitAngles [SimTalk]

setEntryAngles [SimTalk]

setExitAngles [SimTalk]

See also

Calculate Angles

Entry Angle Table

Exit Angle Table

getDestination [SimTalk] - Turntable

Returns the target object to which the *Turntable* designated by <Path> moves the *MU*.

Remarks

You will usually use *getDestination* in the **Target Control**/*TargetCtrl*.

Type

Method

Syntax

```
<Path>.getDestination → object
```

Return Value

The return value has the data type *object*.

Example

```
MyTurntable := ?.getDestination
```

SimTalk

setDestination [SimTalk] - Turntable

TargetCtrl [SimTalk] - Turntable

See also

Target Control [Turntable]

getEntryAngles [SimTalk]

Returns the **Entry Angle Table** of the *Turntable* designated by <Path> and writes it into a table.

Type

Method

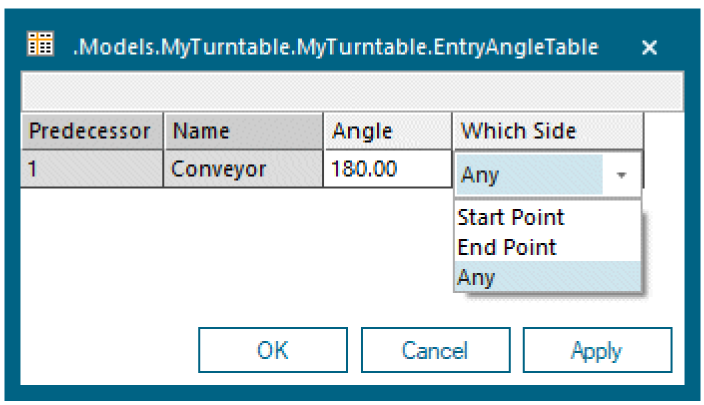
Syntax

```
<Path>.getEntryAngles(EntryAnglesTable:table)
```

Parameter

The parameter *EntryAnglesTable* of data type *table* designates the name of the table into which the *Method* writes the entry angles.

The table contains the number of the **Predecessor**, its **Name**, at which **Angle** the *Connector* from this predecessor connects to the *Turntable*, and **Which Side** turns toward the predecessor.



Example

```
MyTurntable.getEntryAngles(myEntryAngleTable)
```

SimTalk

setEntryAngles [SimTalk]

calculateAngles [SimTalk] - Turntable

See also

Entry Angle Table

getExitAngles [SimTalk]

Returns the **Exit Angle Table** of the *Turntable* designated by <Path> and writes it into a table.

Type

Method

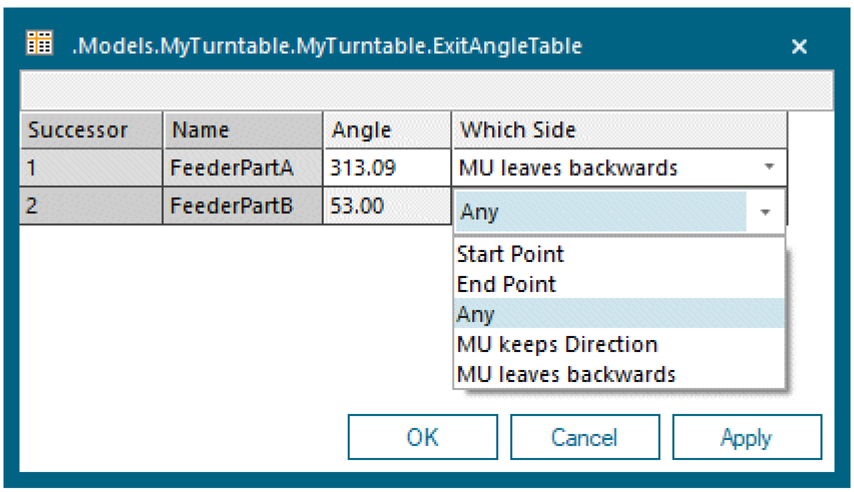
Syntax

```
<Path>.getExitAngles(ExitAnglesTable:table)
```

Parameter

The parameter *ExitAnglesTable* of data type *table* designates the name of the table.

The table contains the number of the **Successor**, its **Name**, at which **Angle** the *Connector* from the *Turntable* connects to this successor, and **Which Side** turns toward the successor.



Example

```
MyTurntable.getExitAngles(myExitAngleTable)
```

SimTalk

setExitAngles [SimTalk]

calculateAngles [SimTalk] - Turntable

See also

Exit Angle Table

goToAngle [SimTalk]

Sets the angle to which the side of the end point of the *Turntable* designated by <Path> rotates.

Remarks

Only applies, if no *MU* is located on it and if no *MU* is waiting for it.

Type

Method

Syntax

```
<Path>.goToAngle(Angle:real)
```

Parameter

The parameter *Angle* of data type *real* designates the angle.

Example

```
?.goToAngle(45)
```

setDestination [SimTalk] - Turntable

Sets the target object to which the *Turntable* designated by <Path> moves the *MU*.

Remarks

You will usually use *setDestination* in the **Target Control**/*TargetCtrl*.

Type

Method

Syntax

```
<Path>.setDestination(Destination:object)
```

Parameter

The parameter *Destination* of data type *object* designates the target object.

Example

```
if @.Name = "PartA"
// the identifier "@" points to the MU, the part in our case,
// which triggers the call of the current method in a control
   ?.setDestination(ConveyorPartA)
   // the identifier "?" points to the object which
   // calls the method, the turntable in our case
else
    ?.setDestination(ConveyorPartB)
end
```

SimTalk

getDestination [SimTalk] - Turntable

TargetCtrl [SimTalk] - Turntable

See also

Target Control [Turntable]

setEntryAngles [SimTalk]

Sets the name of the **Entry Angle Table** that is going to be assigned to the *Turntable* designated by <Path>.

Type

Method

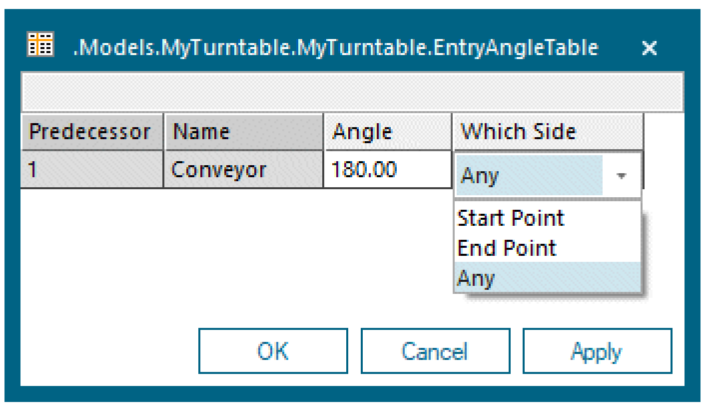
Syntax

```
<Path>.setEntryAngles(EntryAnglesTable:table)
```

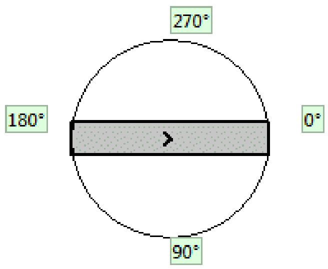
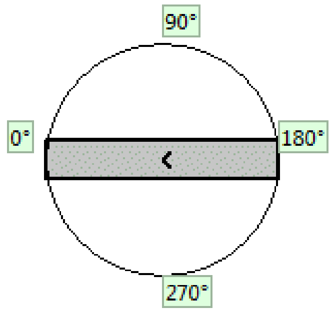
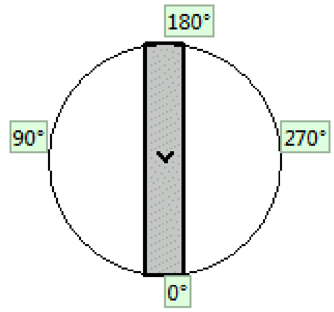
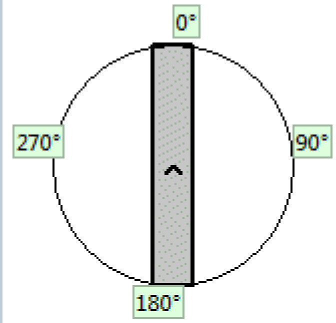
Parameter

The parameter *EntryAnglesTable* of data type *table* designates the name of the table.

It contains the number of the **Predecessor**, the **Name** of the **Predecessor**, the **Angle** at which the *Connector* from this predecessor connects to the *Turntable*, and at **Which Side** it turns toward the predecessor.



The figure illustrates the position of the **Entry Angle** depending on the insertion direction of the *Turntable*.

Insertion direction left to right	Insertion direction right to left	Insertion direction top to bottom	Insertion direction bottom to top
			

Example

```
MyTurntable.setEntryAngles(myEntryAngleTable)
```

SimTalk

getEntryAngles [SimTalk]

calculateAngles [SimTalk] - Turntable

See also

Entry Angle Table

setExitAngles [SimTalk]

Sets the name of the **Exit Angle Table** that is going to be assigned to the *Turntable* designated by <Path>.

Type

Method

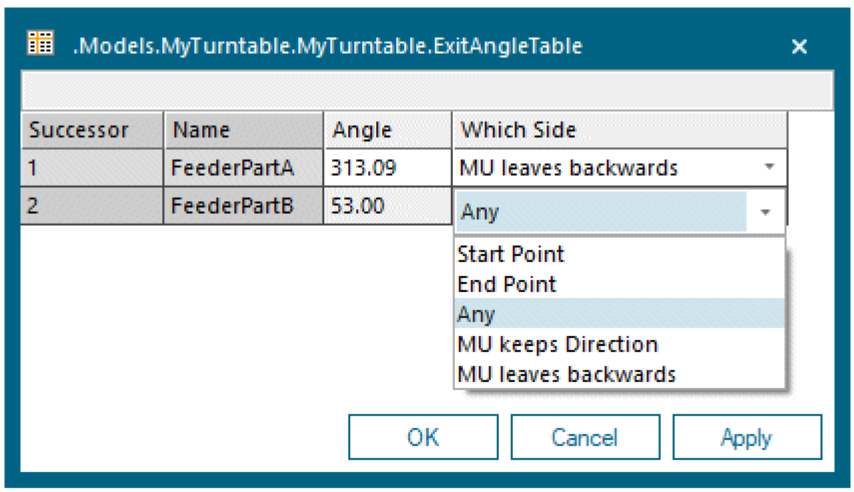
Syntax

```
<Path>.setExitAngles(ExitAnglesTable:table)
```

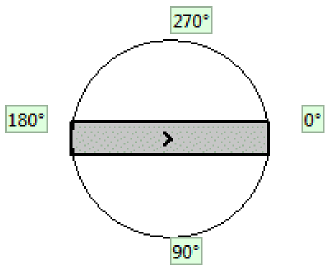
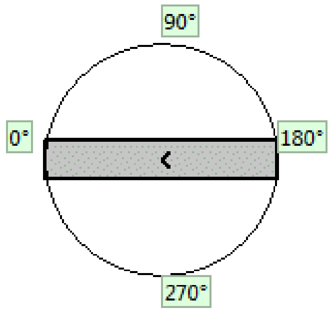
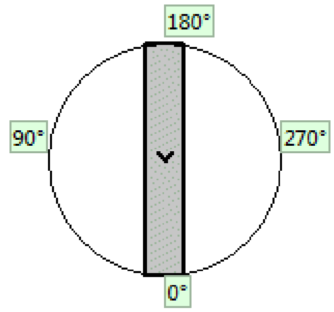
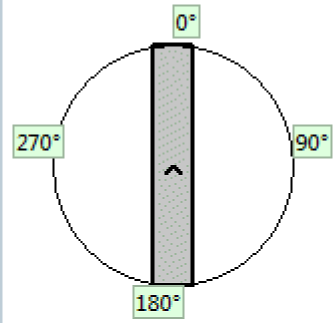
Parameter

The parameter *ExitAnglesTable* of data type *table* designates the name of the table.

It contains the number of the **Successor**, the **Name** of the **Successor**, the **Angle** the *Connector* from the *Turntable* connects to this successor, and at **Which Side** it turns toward the successor.



The figure illustrates the position of the **Exit Angle** depending on the insertion direction of the *Turntable*.

Insertion direction left to right	Insertion direction right to left	Insertion direction top to bottom	Insertion direction bottom to top
			

Example

```
MyTurntable.setExitAngles(myExitAngleTable)
```

SimTalk

getExitAngles [SimTalk]

calculateAngles [SimTalk] - Turntable

See also

Exit Angle Table

stopMU [SimTalk]

Stops the *MU* while the *Turntable* designated by <Path> is still rotating.

Remarks

When the rotation is finished, the *MU* automatically moves again.

Use *stopMU* to stop the *MU* before it reaches the end of the table to prevent collisions for example. You will mostly use *stopMU* in a sensor control.

Type

Method

Syntax

```
<Path>.stopMU
```

Example

```
MyTurntable.stopMU
```

Read-Only Attributes of the Turntable

_Read-Only Attributes of the Turntable

The *Turntable* provides:

- The *read-only attributes* listed in the table of contents to the left.
- The **_Read-Only Attributes of All Objects**.
- The **Read-Only Attributes of the Material Flow Objects**.

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.